République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Université des Sciences et de la Technologie Houari Boumediene
Faculté d'Informatique · Département SI

Master 1 Systèmes d'Information et d'Intelligence Artificielle

Module : **RCR1 — Représentation des Connaissances et Raisonnement**

Implémentation de Formalismes de Représentation des Connaissances et Raisonnement

Logique Modale · Logique des Défauts · Logique Classique
Logique de Description · Réseaux Sémantiques

Domaines d'application : **Industrie musicale algérienne** et **Système judiciaire**

Réalisé par : Dif Arslan Aris
Ziane Berroudja Tesnime

Encadré par : Hadja Faiza Khellaf-Haned

Groupe : 02

TweetyProject 1.27 · OWL API 4.5 · HermiT 1.4.3 · JavaFX 17 · Java 11 · Maven

Année universitaire 2025 – 2026

Table des matières

1 Introduction **2**

2 Outils et bibliothèques **2**

3 Justification des choix **2**

4 TP 1 — Logique Modale **2**

5 TP 2 — Logique des Défauts **5**

6 TP 3 — Logique Classique (Propositionnelle + Prédicats) **7**

7 TP 4 — Logique de Description **9**

8 TP 5 — Réseaux Sémantiques **12**

9 Tableau comparatif **14**

Références **14**

Liste des tableaux

1 Extensions calculées par artiste — domaine musical 6
2 Extensions calculées — domaine judiciaire 7
3 Comparaison des cinq formalismes 14
4 Table des références bibliographiques 14

Table des figures

1 Modèle de Kripke — domaine musical 3
2 Modèle de Kripke — domaine judiciaire 4
3 Réseau sémantique — domaine musical 12

1 Introduction

La **Représentation des Connaissances et Raisonnement** (RCR) vise à formaliser un domaine pour qu'un système puisse raisonner automatiquement. Contrairement à la programmation procédurale, l'approche déclarative décrit *ce que l'on sait* et laisse un moteur d'inférence dériver les conclusions.

Ce projet implémente cinq formalismes en Java et les applique aux mêmes deux domaines :

— **Industrie musicale algérienne** — artistes (Soolking, Feu, Tif, Flenn), labels (Universal, Believe), albums, certifications, statuts contractuels.

— **Système judiciaire** — magistrats, avocats, accusés (mineur, récidiviste), infractions, tribunaux ; domaine riche en exceptions et conflits de règles.

2 Outils et bibliothèques

Outil	Version	Rôle
Java JDK	11	Langage de programmation
Maven	3.6+	Gestion de projet
TweetyProject	1.27	Logique modale, défauts, classique, réseaux sémantiques
OWL API	4.5.29	Manipulation d'ontologies OWL 2
HermiT	1.4.3.456	Raisonneur OWL DL
SAT4J	(inclus)	Solveur SAT propositionnel
JavaFX	17.0.2	Interface graphique

3 Justification des choix

— **TweetyProject** : bibliothèque Java unifiée couvrant plusieurs formalismes, idéale pour comparer leur expressivité dans un seul projet.

— **OWL API + HermiT** : standard W3C pour les ontologies ; HermiT garantit complétude et correction pour SROIQ.

— **SAT4J** : intégré à Tweety, efficace en pratique malgré NP-complet.

— **Les deux domaines** permettent de mettre en évidence les spécificités de chaque formalisme : hiérarchies, exceptions, états alternatifs, contraintes dures.

4 TP 1 — Logique Modale

Domaines

— **Musical** : 4 mondes — label (w_1), fan (w_2), artiste (w_3), critique (w_4).

— **Judiciaire** : 5 mondes — juge (w_1), avocat (w_2), procureur (w_3), témoin (w_4), accusé (w_5).

Lois sémantique de Kripke

Un modèle de Kripke est un triplet $\mathcal{M} = \langle W, R, V \rangle$. Les cas modaux :

$$\mathcal{M}, w \models \Box\varphi \iff \forall v,\ (w,v) \in R \Rightarrow \mathcal{M}, v \models \varphi \qquad \mathcal{M}, w \models \Diamond\varphi \iff \exists v,\ (w,v) \in R \wedge \mathcal{M}, v \models \varphi$$

Propriétés de R et systèmes modaux associés :

— Réflexive $\Rightarrow$ axiome $\Box p \to p$ — système **T**

— Transitive $\Rightarrow$ axiome $\Box p \to \Box\Box p$ — système **S4**

— Symétrique ⇒ axiome $p \to \Box\Diamond p$ — système **B**
— Euclidienne ⇒ axiome $\Diamond p \to \Box\Diamond p$ — système **S5**
— Sérielle ⇒ axiome $\Box p \to \Diamond p$ — système **D**

Graphe de Kripke — Domaine musical

Chaque nœud contient les atomes **vrais** dans ce monde. Les arcs représentent la relation d'accessibilité R.

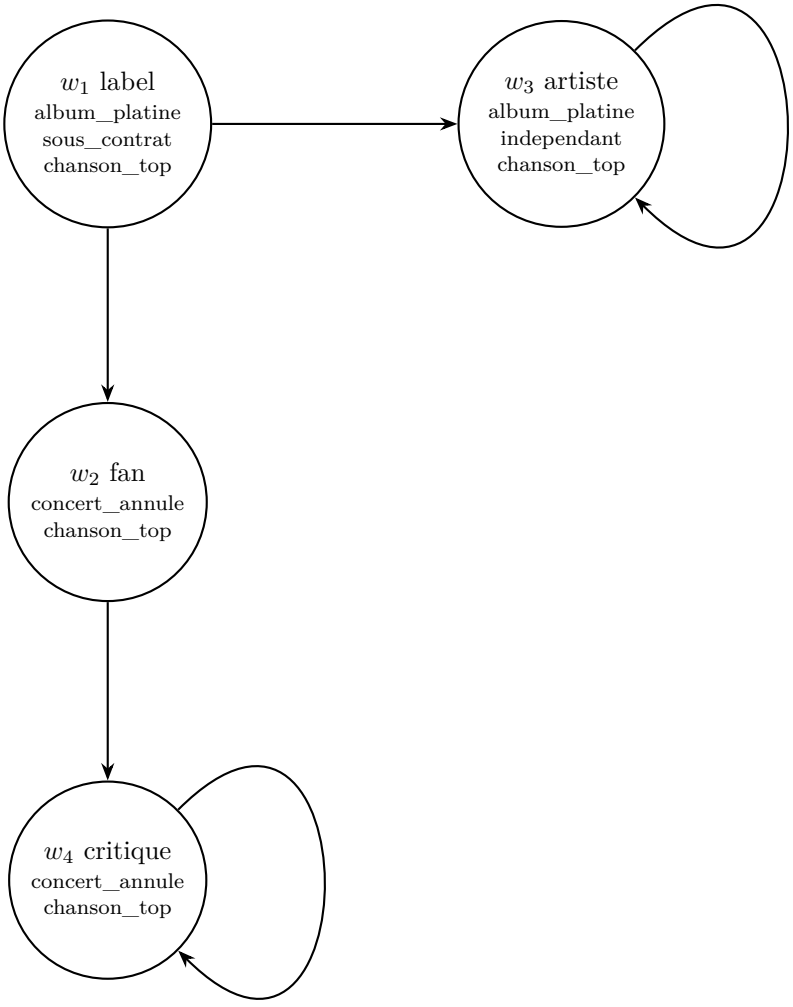


FIGURE 1 – Modèle de Kripke — domaine musical

Graphe de Kripke — Domaine judiciaire

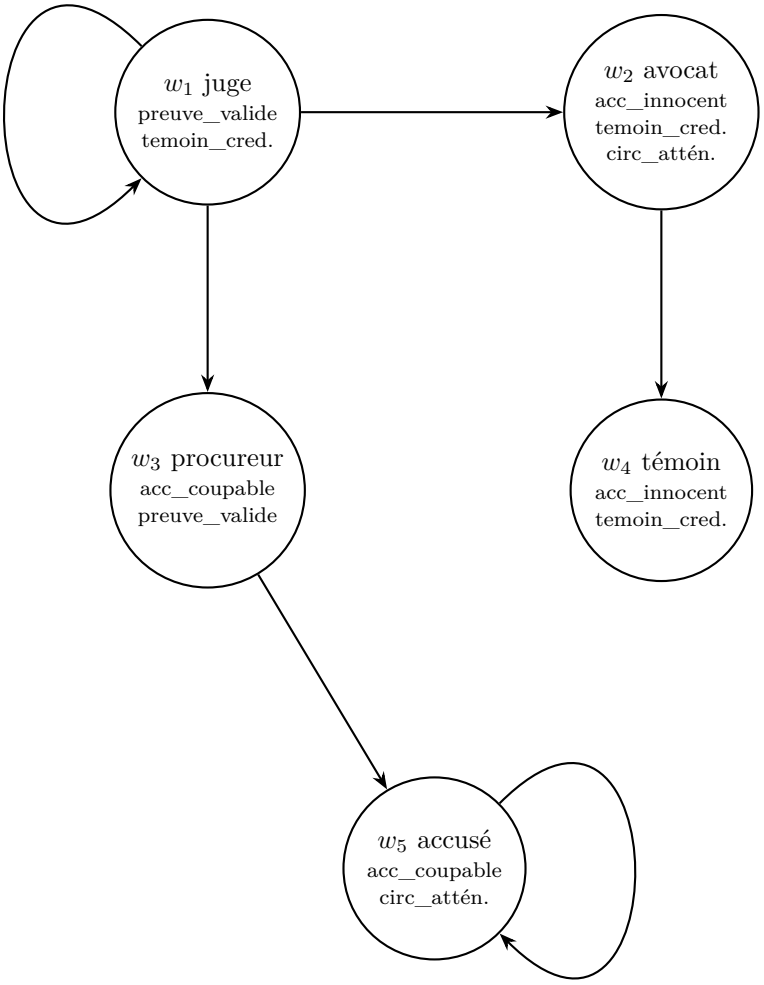


FIGURE 2 – Modèle de Kripke — domaine judiciaire

Exploitations

Domaine musical :

- $\Box$ `album_platine` en w_1 : **FAUX** — $w_2 \in \text{Succ}(w_1)$ mais album_platine faux en w_2
- $\Diamond$ `concert_annule` en w_1 : **VRAI** — vrai en $w_2 \in \text{Succ}(w_1)$
- $\Box$ `chanson_top` en w_2 : **VRAI** — $\text{Succ}(w_2) = \{w_4\}$, chanson_top vrai en w_4
- $\Diamond$ `artiste_independant` en w_1 : **VRAI** — vrai en $w_3 \in \text{Succ}(w_1)$
- $\Box$(`album_platine` $\rightarrow$ `sous_contrat`) en w_1 : **FAUX** — en w_3 : album=V, contrat=F

Domaine judiciaire :

- $\Box$ `preuve_valide` en w_1 : **FAUX** — faux en w_2
- $\Diamond$ `accuse_innocent` en w_1 : **VRAI** — vrai en w_2
- $\Box$ `accuse_coupable` en w_3 : **VRAI** — $\text{Succ}(w_3) = \{w_5\}$, coupable vrai
- $\Box$ `temoin_credible` en w_2 : **VRAI** — $\text{Succ}(w_2) = \{w_4\}$, crédible vrai

Extension (propriétés de R)

— Modèle musical : non réflexive, non transitive, non symétrique, **sérielle**.
— Modèle judiciaire : réflexive en w_1 seulement, non symétrique, non transitive, **non sérielle** (w_4 sans successeur).

Extension temporelle

La relation R devient un *ordre temporel* : les mondes sont des instants. On ajoute quatre opérateurs :

$$\mathbf{G} = \Box_{\text{futur}} \quad \mathbf{F} = \Diamond_{\text{futur}} \quad \mathbf{H} = \Box_{\text{passé}} \quad \mathbf{P} = \Diamond_{\text{passé}}$$

Musique — carrière de Soolking : t_1 (2017, indépendant) $< t_2$ (signe Universal) $< t_3$ (sort *Guerilla*) $< t_4$ (album platine) $< t_5$ (nouvel album).
Justice — déroulement d'un procès : t_1 (infraction) $< t_2$ (arrestation) $< t_3$ (inculpation) $< t_4$ (audience) $< t_5$ (verdict) $< t_6$ (libération).

Valuation V (judiciaire) :
`libre` $= \{t_1, t_6\}$, `en_detention` $= \{t_2, t_3, t_4, t_5\}$, `accuse` $= \{t_3, t_4\}$, `juge` $= \{t_4, t_5\}$, `condamne` $= \{t_5, t_6\}$, `presume_innocent` $= \{t_1, t_2, t_3, t_4\}$, `droit_avocat` $= \{t_2, t_3, t_4, t_5, t_6\}$.

Requêtes temporelles — domaine judiciaire :

#	Formule	Instant	Résultat
Q1	**G** libre	t_6	**VRAI** (futur vide ⇒ vrai)
Q2	**F** condamne	t_1	**VRAI** (verdict en t_5)
Q3	**H** libre	t_3	**FAUX** (détention dès t_2)
Q4	**P** libre	t_5	**VRAI** (libre en t_1)
Q5	**H** presume_innocent	t_4	**VRAI** (vrai avant t_4)
Q6	**G** presume_innocent	t_5	**FAUX** (faux en t_6)
Q7	**F** libre	t_3	**VRAI** (libéré en t_6)
Q8	**G** droit_avocat	t_2	**VRAI** (garanti t_3–t_6)

Implémentation

Modale classique : packages `rcr.modal` avec `ModalLogicMusic` et `ModalLogicJustice`. L'évaluation est récursive sur la structure de la formule (`Necessity`, `Possibility`, `Implication`…) via les types `MlHerbrandInterpretation` (mondes) et `AccessibilityRelation` de Tweety. Extension temporelle : sous-package `rcr.modal.temporal`, évaluation en Java pur (méthodes statiques `G(atom, t)`, `F`, `H`, `P`) sur une liste ordonnée d'instants — Tweety ne fournit pas G/F/H/P nativement.

5 TP 2 — Logique des Défauts

Domaine

Musical : statuts contractuels et droits des artistes (Soolking, DZ_Master, Atlas).
Judiciaire : présomption d'innocence, récidive, tribunal compétent (Ali, Omar, Karim).

Lois

Une théorie des défauts $\Delta = (W, D)$: W = faits certains, D = défauts.

$$\frac{\alpha(x) : \beta(x)}{\gamma(x)} \quad \alpha = \text{prérequis}, \quad \beta = \text{justification}, \quad \gamma = \text{conclusion}$$

Une **extension** E est le plus petit ensemble clos par déduction tel que $W \subseteq E$ et tout défaut applicable est appliqué. En cas de conflit → plusieurs extensions.
Sémantiques : **crédule** (∃ une extension) / **sceptique** (toutes les extensions).

Défauts — Domaine musical

$$d_1 : \frac{\text{EstArtiste}(x) : \text{LibreDroits}(x)}{\text{LibreDroits}(x)}$$

$$d_2 : \frac{\text{SigneLabel}(x) : \neg\text{LibreDroits}(x)}{\neg\text{LibreDroits}(x)}$$

$$d_3 : \frac{\text{EstArtiste}(x) \wedge \text{AlbumSorti}(x) : \text{Royalties}(x)}{\text{Royalties}(x)}$$

$$d_4 : \frac{\text{SigneLabel}(x) : \text{PartageRoyalties}(x)}{\text{PartageRoyalties}(x)}$$

$$d_5 : \frac{\text{IndependantLongtemps}(x) : \text{ExperienceSolo}(x)}{\text{ExperienceSolo}(x)}$$

$$d_6 : \frac{\text{NouveauContrat}(x) : \neg\text{LibreDroits}(x)}{\neg\text{LibreDroits}(x)}$$

$$d_7 : \frac{\text{PlusDeDeuxAns}(x) \wedge \text{AlbumSorti}(x) : \text{AlbumCertifie}(x)}{\text{AlbumCertifie}(x)}$$

Résultats par artiste — domaine musical

Table 1 – Extensions calculées par artiste — domaine musical

Artiste	**Faits** W	**Ext.**	**Conflit**
Soolking	EstArtiste + SigneLabel + AlbumSorti	2	d_1 vs d_2
DZ_Master	EstArtiste + IndepLongtemps + NouveauContrat	2	d_1 vs d_6
Atlas	EstArtiste + AlbumSorti + PlusDeDeuxAns	1	—
Atlas (après)	EstArtiste + AlbumSorti + PlusDeDeuxAns + SigneLabel	2	d_1 vs d_2

Conflit entre d_1 et d_2 (Soolking) :
d_1 conclut *LibreDroits* et d_2 conclut $\neg$*LibreDroits*. Les deux défauts sont utilisables et leurs conséquents sont contradictoires. Les défauts sont alors mutuellement exclusifs : l'applicabilité de d_1 rend d_2 non applicable et inversement. La théorie admet donc deux extensions :

— $E_1 = TH(\{\ldots,\ \textit{LibreDroits},\ \textit{Royalties},\ \textit{PartageRoyalties}\})$
— $E_2 = TH(\{\ldots,\ \neg\textit{LibreDroits},\ \textit{Royalties},\ \textit{PartageRoyalties}\})$

Les formules *Royalties* et *PartageRoyalties* appartiennent aux deux extensions. La formule *LibreDroits* appartient uniquement à E_1.

Démonstration de la non-monotonie

Atlas avant ajout de SigneLabel : d_1 s'applique seul ⇒ extension unique avec LibreDroits $\in E$.

Atlas après ajout de SigneLabel : d_2 déclenche un conflit ⇒ 2 extensions, LibreDroits $\notin \bigcap E_i$.
Enrichir W d'un seul fait a *invalidé* une conclusion antérieure ⇒ **non-monotonie**.

Défauts — Domaine judiciaire

$$d_1 : \frac{\text{Accuse}(x) : \text{Innocent}(x)}{\text{Innocent}(x)}$$

$$d_2 : \frac{\text{Recidiviste}(x) : \text{PeineLourde}(x)}{\text{PeineLourde}(x)}$$

$$d_3 : \frac{\text{Mineur}(x) : \text{TribunalEnfants}(x)}{\text{TribunalEnfants}(x)}$$

$$d_4 : \frac{\text{CrimeGrave}(x) : \neg\text{TribunalEnfants}(x)}{\neg\text{TribunalEnfants}(x)}$$

$$d_5 : \frac{\text{Temoignage}(x) : \text{Acquittement}(x)}{\text{Acquittement}(x)}$$

$$d_6 : \frac{\text{Recidiviste}(x) : \neg\text{Innocent}(x)}{\neg\text{Innocent}(x)}$$

Résultats par individu — domaine judiciaire

Table 2 – Extensions calculées — domaine judiciaire

Individu	**Faits** W	**Ext.**	**Conflit**
Ali	Accuse + Temoignage	1	—
Omar	Accuse + Recidiviste	2	d_1 vs d_6
Karim	Accuse + Mineur + CrimeGrave	2	d_3 vs d_4

Conflit entre d_3 et d_4 (Karim) :
d_3 conclut *TribunalEnfants* (car *Mineur*) et d_4 conclut $\neg$*TribunalEnfants* (car *CrimeGrave*). Les deux défauts sont utilisables et leurs conséquents sont contradictoires. Les défauts sont alors mutuellement exclusifs, ce qui conduit à deux extensions possibles. La formule *Innocent*, issue de d_1 et sans opposition, appartient aux deux extensions.

Implémentation

Package `rcr.defaut`. Construction de la théorie sous forme de chaîne parsée par `RdlParser` ; calcul des extensions par `SimpleDefaultReasoner.getModels(dt.ground())` qui implémente l'algorithme original de Reiter. **Choix critique** : on construit *une théorie par individu* pour éviter l'explosion combinatoire du `DefaultProcessTree` sur le grounding global.

6 TP 3 — Logique Classique (Propositionnelle + Prédicats)

Domaine

Le même quatuor d'artistes (*Soolking, Feu, Tif, Flenn*) est modélisé deux fois : une fois en propositionnel (atomes plats), une fois en prédicats (axiomes universels). Côté justice : trois accusés (*Ali, Omar, Karim*).

Lois propositionnelles (CNF + SAT4J) — musique

Faits (clauses unitaires) : `soolking`, `feu`, `tif`, `flenn`, `signe_soolking`, `signe_feu`, `indep_feu`.

Clauses dérivationnelles (toutes) :

$$c_1 = \neg\text{soolking} \vee \neg\text{signe_soolking} \vee \text{partage_soolking}$$
$$c_2 = \neg\text{feu} \vee \neg\text{signe_feu} \vee \text{partage_feu}$$
$$c_3 = \neg\text{tif} \vee \text{royalties_tif}$$
$$c_4 = \neg\text{flenn} \vee \text{royalties_flenn}$$
$$c_5 = \neg\text{indep_feu} \vee \text{expsolo_feu}$$

Lois propositionnelles — justice

Faits : `ali`, `omar`, `karim`, `temoignage_ali`, `recidiviste_omar`, `mineur_karim`, `crime_grave_karim`.
Clauses :

$$c_1 = \neg\text{ali} \vee \text{innocent_ali} \quad \text{(présomption d'innocence)}$$
$$c_2 = \neg\text{ali} \vee \neg\text{temoignage_ali} \vee \text{acquitte_ali}$$
$$c_3 = \neg\text{omar} \vee \text{innocent_omar}$$
$$c_4 = \neg\text{recidiviste_omar} \vee \text{peine_lourde_omar}$$
$$c_5 = \neg\text{mineur_karim} \vee \text{tribunal_enfants_karim}$$
$$c_6 = \neg\text{crime_grave_karim} \vee \neg\text{tribunal_enfants_karim} \quad \text{(opt., crée Q6)}$$

L'entailment $\text{KB} \models \varphi$ est résolu par **réfutation** : $\text{KB} \models \varphi$ ssi $\text{KB} \cup \{\neg\varphi\}$ est UNSAT.

Lois prédicats (chaînage avant) — musique

Domaine fini {soolking, feu, tif, flenn}. Axiomes universels (tous) :

$$\text{ax}_1 : \forall x\,(\text{EstRappeurAlgerien}(x) \Rightarrow \text{EstRappeur}(x))$$
$$\text{ax}_2 : \forall x\,(\text{EstRappeur}(x) \Rightarrow \text{EstArtiste}(x))$$
$$\text{ax}_3 : \forall x\,(\text{EstArtiste}(x) \Rightarrow \text{EstMusicien}(x))$$
$$\text{ax}_4 : \forall x\,(\text{EstArtiste}(x) \wedge \text{AlbumSorti}(x) \Rightarrow \text{RecitRoyalties}(x))$$
$$\text{ax}_5 : \forall x\,(\text{SigneLabel}(x) \Rightarrow \text{PartageRoyalties}(x))$$
$$\text{ax}_6 : \forall x\,(\text{IndependantLongtemps}(x) \Rightarrow \text{ExperienceSolo}(x))$$
$$\text{ax}_7 : \forall x\,(\text{PlusDeDeuxAns}(x) \wedge \text{AlbumSorti}(x) \Rightarrow \text{AlbumCertifie}(x))$$

Faits initiaux : `EstRappeurAlgerien` pour les 4 ; `SigneLabel` pour soolking, feu ; `AlbumSorti` pour les 4 ; `IndependantLongtemps(feu)` ; `PlusDeDeuxAns` pour tif, flenn.

Lois prédicats — justice

Domaine fini {ali, omar, karim}. Axiomes universels (tous) :

$$\text{ax}_1 : \forall x\,(\text{EstAccuse}(x) \Rightarrow \text{DroitAuProces}(x))$$
$$\text{ax}_2 : \forall x\,(\text{EstAccuse}(x) \Rightarrow \text{DroitDefense}(x))$$
$$\text{ax}_3 : \forall x\,(\text{ATemoignage}(x) \Rightarrow \text{CandidatAcquittement}(x))$$
$$\text{ax}_4 : \forall x\,(\text{EstRecidiviste}(x) \Rightarrow \text{PeineLourde}(x))$$
$$\text{ax}_5 : \forall x\,(\text{EstMineur}(x) \Rightarrow \text{TribunalEnfants}(x))$$
$$\text{ax}_6 : \forall x\,(\text{CrimeGrave}(x) \Rightarrow \text{SupJuridiction}(x))$$
$$\text{ax}_7 : \forall x\,(\text{PeineLourde}(x) \Rightarrow \text{SuiviRenforce}(x))$$
$$\text{ax}_8 : \forall x\,(\text{TribunalEnfants}(x) \wedge \text{CrimeGrave}(x) \Rightarrow \text{NecessiteAvocatPublic}(x))$$

Faits initiaux : `EstAccuse` pour les 3 ; `ATemoignage(ali)` ; `EstRecidiviste(omar)` ; `EstMineur(karim)` ; `CrimeGrave(karim)`.

Exploitation (toutes les requêtes)

Propositionnel musique :

Requête	Résultat
Q1 : KB satisfiable ?	**VRAI**
Q2 : KB $\models$ partage_soolking	**VRAI** (faits + c_1)
Q3 : KB $\models$ expsolo_feu	**VRAI** (faits + c_5)
Q4 : KB +{signe_flenn} $\models$ partage_flenn	**VRAI** (monotonie)
Q5 : KB $\cup$\{¬royalties_flenn\} satisfiable ?	**FAUX** (contradiction)

Propositionnel justice :

Requête	Résultat
Q1 : KB (sans c_6) satisfiable ?	**VRAI**
Q2 : KB $\models$ acquitte_ali	**VRAI** (ali + temoignage + c_2)
Q3 : KB $\models$ innocent_ali	**VRAI** (ali + c_1)
Q4 : KB $\models$ peine_lourde_omar	**VRAI** (récidiviste + c_4)
Q5 : KB $\models$ innocent_omar	**VRAI** (omar + c_3)
Q6 : KB $\cup\{c_6\}$ satisfiable ?	**FAUX** (conflit karim)
Q7 : KB $\cup$\{¬innocent_omar\} sat. ?	**FAUX** (monotonie)

Prédicats musique :

Requête	Résultat
Q1 : $\forall x\,(\text{EstRappeurAlgerien}(x) \Rightarrow \text{RecitRoyalties}(x))$	**VRAI** (chaîne ax_1+ax_2+ax_4)
Q2 : KB $\models$ PartageRoyalties(soolking)	**VRAI** (ax_5)
Q3 : KB $\models$ ExperienceSolo(feu)	**VRAI** (ax_6)
Q4 : KB $\models$ AlbumCertifie(tif)	**VRAI** (ax_7)
Q5 : $\exists x\,\text{ExperienceSolo}(x)$	**VRAI** (témoin : feu)
Q6 : KB $\models$ EstMusicien(soolking)	**VRAI** (chaîne ax_1+ax_2+ax_3)

Prédicats justice : analogues, démontrant la chaîne `EstAccuse` $\rightarrow$ `DroitDefense` (ax_2), la peine renforcée (ax_4+ax_7), et la conjonction `Mineur` $\wedge$ `CrimeGrave` $\rightarrow$ `NecessiteAvocatPublic` (ax_5+ax_8 pour karim).

Implémentation

Package `rcr.classical`. Pour la propositionnelle, méthodes statiques publiques `buildKB()` (retourne un `PlBeliefSet`), `entails()`, `isSatisfiable()` ; le raisonneur est `SatReasoner` de Tweety configuré sur `Sat4jSolver`. Pour les prédicats : forward chaining naïf en Java pur sur `Set<String>` de faits ; méthode `deriverTout()` exécute la boucle jusqu'au point fixe.

7 TP 4 — Logique de Description

Domaine

Ontologie musicale complète avec hiérarchie de classes (`Artiste` $\sqsubseteq$ `Personne`, `Rappeur` $\sqsubseteq$ `Artiste`, etc.), rôles (`signe_chez`, `produit`, `reçoit`) et 10 individus :

Individu	Type asserté	Rôle
Soolking, Feu, Tif, Flenn	RappeurAlgerien	Rappeurs algériens
Universal	MajorLabel	Major label
Believe	IndependantLabel	Label indépendant
Guerilla, Dernière Heure	Album	Albums produits
Cert_Or, Cert_Platine	CertificationOr/Pl.	Certifications de ventes

TBox musique

$$
\begin{aligned}
\text{Artiste} &\equiv \text{Personne} \sqcap \exists\, \text{produit.Oeuvre} \\
\text{Rappeur} &\equiv \text{Artiste} \sqcap \exists\, \text{pratique.Rap} \sqcap \forall\, \text{produit.TitreRap} \\
\text{RappeurAlgerien} &\equiv \text{Rappeur} \sqcap \exists\, \text{chante_en.Darija} \\
\text{Chanteur} &\equiv \text{Artiste} \sqcap \exists\, \text{interprete.Chanson} \\
\text{DJ} &\equiv \text{Artiste} \sqcap \exists\, \text{mixe.Titre} \\
\text{Compositeur} &\equiv \text{Artiste} \sqcap \exists\, \text{compose.Oeuvre} \\
\text{Label} &\equiv \text{Organisation} \sqcap \exists\, \text{distribue.Album} \\
\text{MajorLabel} &\equiv \text{Label} \sqcap \exists\, \text{signe.PlusDe100Artistes} \\
\text{IndependantLabel} &\equiv \text{Label} \sqcap \neg\, \text{MajorLabel} \\
\text{Album} &\equiv \text{Oeuvre} \sqcap \exists\, \text{contient.Titre} \sqcap \exists\, \text{produit_par.Artiste} \\
\text{AlbumCertifie} &\equiv \text{Album} \sqcap \exists\, \text{reçoit.Certification} \\
\text{AlbumOr} &\equiv \text{AlbumCertifie} \sqcap \exists\, \text{reçoit.CertificationOr} \\
\text{AlbumPlatine} &\equiv \text{AlbumCertifie} \sqcap \exists\, \text{reçoit.CertificationPlatine} \\
\text{ArtisteSigne} &\equiv \text{Artiste} \sqcap \exists\, \text{signe_chez.Label} \\
\text{ArtisteMajor} &\equiv \text{Artiste} \sqcap \exists\, \text{signe_chez.MajorLabel} \\
\text{ArtisteIndependant} &\equiv \text{Artiste} \sqcap \neg\, \exists\, \text{signe_chez.MajorLabel} \\
\text{ArtisteLibreDroits} &\equiv \text{Artiste} \sqcap \neg\, \exists\, \text{signe_chez.Label} \\
\text{Featuring} &\equiv \text{Titre} \sqcap \exists\, \text{implique.Artiste} \sqcap \exists\, \text{implique.ArtisteEtranger} \\
\text{ArtisteAvecRoyalties} &\equiv \text{Artiste} \sqcap \exists\, \text{produit.Album} \\
\text{ArtistePartageRoyalties} &\equiv \text{ArtisteAvecRoyalties} \sqcap \exists\, \text{signe_chez.Label}
\end{aligned}
$$

Inclusions : $\texttt{Titre} \sqsubseteq \texttt{Oeuvre}$, $\texttt{TitreRap} \sqsubseteq \texttt{Titre}$, $\texttt{MajorLabel} \sqsubseteq \texttt{Label}$, $\texttt{CertificationOr} \sqsubseteq \texttt{Certification}$, $\texttt{CertificationPlatine} \sqsubseteq \texttt{Certification}$.

ABox musique

RappeurAlgerien :	soolking, feu, tif, flenn
MajorLabel :	universal
IndependantLabel :	believe
Album :	guerilla, derniere_heure
CertificationOr :	cert_or CertificationPlatine : cert_platine
ArtisteLibreDroits :	tif, flenn (*assertion explicite, OWA local*)
`signe_chez` :	(soolking, universal), (feu, believe)
`produit` :	(soolking, guerilla), (feu, derniere_heure), (tif, guerilla)
`reçoit` :	(guerilla, cert_or), (derniere_heure, cert_platine)

TBox justice (définitions principales)

$$
\begin{aligned}
\text{ActeurJudiciaire} &\equiv \text{Personne} \sqcap \exists\, \text{participe_a.Proces} \\
\text{Magistrat} &\equiv \text{ActeurJudiciaire} \sqcap \exists\, \text{siege_a.Tribunal} \\
\text{Juge} &\equiv \text{Magistrat} \sqcap \exists\, \text{preside.Audience} \\
\text{Procureur} &\equiv \text{Magistrat} \sqcap \exists\, \text{requiert.Peine} \\
\text{Avocat} &\equiv \text{ActeurJudiciaire} \sqcap \exists\, \text{defend.Accuse} \\
\text{Accuse} &\equiv \text{Personne} \sqcap \exists\, \text{fait_objet.Inculpation} \\
\text{Crime} &\equiv \text{Infraction} \sqcap \exists\, \text{puni_par.PeineReclusion} \\
\text{Delit} &\equiv \text{Infraction} \sqcap \exists\, \text{puni_par.PeineEmprisonnement} \\
\text{CrimeGrave} &\equiv \text{Crime} \sqcap \exists\, \text{cause.PrejudiceGrave} \\
\text{CourAssises} &\equiv \text{Tribunal} \sqcap \forall\, \text{juge.Crime} \\
\text{TribunalEnfants} &\equiv \text{Tribunal} \sqcap \forall\, \text{juge.InfractionMineur} \\
\text{AccuseMineur} &\equiv \text{Accuse} \sqcap \exists\, \text{a_age.MoinsDe18Ans} \\
\text{AccuseRecidiviste} &\equiv \text{Accuse} \sqcap \exists\, \text{a_antecedent.Condamnation} \\
\text{AccusePresumInnocent} &\equiv \text{Accuse} \sqcap \neg\, \text{Condamne} \\
\text{AccuseAvecTemoignage} &\equiv \text{Accuse} \sqcap \exists\, \text{beneficie.TemoignageFavorable} \\
\text{CasSimple} &\equiv \text{Accuse} \sqcap \neg\, \text{AccuseRecidiviste} \sqcap \neg\, \text{AccuseMineur} \\
\text{CasComplexe} &\equiv \text{Accuse} \sqcap (\text{AccuseRecidiviste} \sqcup \text{AccuseMineur})
\end{aligned}
$$

ABox justice

Accuse :	ali, omar, karim
Juge :	juge_salem Avocat : avocat_riad Procureur : procureur_malik
TemoignageFav. :	temoignage_ali
Condamnation :	antecedent_omar
CrimeGrave :	crime_karim
beneficie :	(ali, temoignage_ali)
a_antecedent :	(omar, antecedent_omar)
fait_objet :	(karim, crime_karim)
juge_par :	(omar, trib_correctionnel), (karim, trib_enfants)

Exploitations (HermiT)

Musicales :

- ArtisteIndependant(soolking) : **FAUX** — soolking $\in \exists$signe_chez.MajorLabel
- ArtisteLibreDroits(feu) : **FAUX** — feu signé chez believe $\sqsubseteq$ Label
- ArtisteLibreDroits(tif) : **VRAI** — asserté en ABox
- AlbumOr(guerilla) : **VRAI** — reçoit(guerilla, certOr)
- instances(ArtisteSigne) = {soolking, feu} ; ArtisteMajor $\sqsubseteq$ ArtisteSigne : **VRAI**

Judiciaires :

- AccusePresumInnocent(ali) : **VRAI**
- CasComplexe(karim) : **VRAI** — karim :AccuseMineur
- instances(CasComplexe) = {omar, karim}
- Disjoint(CasSimple, CasComplexe) : **VRAI** ; Disjoint(Juge, Procureur) : **FAUX** — non déclarés disjoints

Implémentation

Package `rcr.description`. La méthode publique `buildOntology()` de `DescriptionLogicMusic` construit l'`OWLOntology` via l'OWL API (création du manager, ajout des axiomes TBox puis ABox), retourne l'objet. La GUI charge cet objet une fois en background, instancie `ReasonerFactory.createReasoner(onto)` et précompile les inférences via `precomputeInferences(CLASS_HIERARCHY, CLASS_ASSERTIONS)`. Les requêtes interactives sont ensuite instantanées car HermiT reste vivant.

8 TP 5 — Réseaux Sémantiques

Domaine

Deux réseaux illustrent la même intuition (héritage avec exceptions) :

— **Musique** : `Artiste` → `ArtisteSigne` → `Soolking` avec un `ArtisteIndep` qui hérite de `Artiste` mais a une exception stricte ¬sous-contrat envers `Label`.

— **Justice** : `PartieProcès` → `Accusé` → `Récidiviste` où `Accusé` hérite de la présomption d'innocence, mais `Récidiviste` la perd par une exception stricte ¬presume.

Lois

Un réseau sémantique est un graphe orienté étiqueté :

— **Nœuds** : concepts (classes) ou individus.
— **Arcs** : relations typées (`is-a`, `instance-of`, propriétés).

`is-a` : Spécialisation/généralisation : Rappeur `is-a` Artiste.

`instance-of` : Appartenance : soolking `instance-of` Rappeur.

Propriétés ; : Relations arbitraires : soolking `signe_chez` universal.

Réseau — Domaine musical

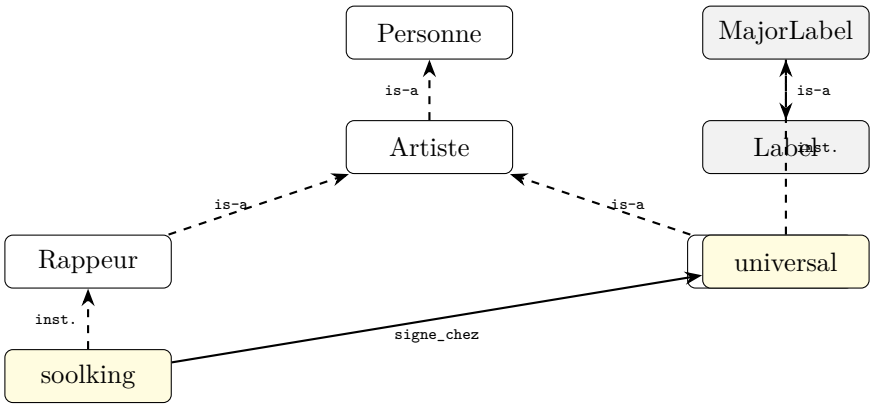


FIGURE 3 – Réseau sémantique — domaine musical

Héritage et propagation de marqueurs

Le mécanisme d'héritage permet à un nœud de récupérer les propriétés de ses ancêtres via les arcs `is-a`.

La **propagation de marqueurs** est la technique algorithmique associée : on marque deux nœuds (M1, M2) puis on propage ces marqueurs vers le haut (sens inverse des arcs `is-a`). À chaque étape, on vérifie si deux nœuds différemment marqués sont reliés par un arc de propriété, ce qui signale une relation indirecte entre les concepts initiaux.

— **Héritage simple** : un seul parent dans la hiérarchie.
— **Avec exceptions** : ne propriété héritée peut être annulée ou redéfinie localement. ⇒ rend le réseau **non-monotone**.

Exemple (musical) : soolking hérite `LibreDroits` d'Artiste. L'ajout de `signe_chez` crée une exception locale ⇒ `LibreDroits` est invalidé.
Exemple (judiciaire) : karim hérite TribunalEnfants d'AccuséMineur. La propriété CrimeGrave crée un conflit avec le chemin normal ⇒ deux chemins d'héritage incompatibles (conflit non résolu sans priorité).

Modalités et négation

Pour représenter les attitudes propositionnelles d'un agent (ce qu'il *sait*, *croit*, *veut*) ainsi que les faits qu'il *nie*, on crée des instances de classes modales dédiées :

Classe modale	Paramètre `agent`	Paramètre `objet`
`Savoir`	l'agent qui sait	le fait su (mini-réseau)
`Croyance`	l'agent qui croit	le fait cru (mini-réseau)
`Négation`	l'agent qui nie	le fait nié
`Volonté`	l'agent qui veut	l'état désiré

Principe clé : l'objet d'une modalité est lui-même un *mini-réseau sémantique*, ce qui justifie le **partitionnement** du graphe en sous-graphes contextuels.

Exemples dans le domaine musical :

— `s1 : Savoir`, `s1.agent = label_universal`, `s1.objet = {soolking signe_chez universal}`
⇒ le label sait que soolking est signé chez lui.
— `c1 : Croyance`, `c1.agent = soolking`, `c1.objet = {soolking instance-of ArtisteLibreDroits}`
⇒ soolking *croit* (à tort) être libre de droits.
— `n1 : Négation`, `n1.objet = {soolking instance-of ArtisteLibreDroits}`
⇒ représentation explicite du fait que soolking n'est pas libre.

Exemples dans le domaine judiciaire :

— `s2 : Savoir`, `s2.agent = juge`, `s2.objet = {omar instance-of AccuséRécidiviste}`
⇒ le juge sait qu'omar est récidiviste.
— `c2 : Croyance`, `c2.agent = avocat`, `c2.objet = {ali instance-of AccuséPresumInnocent}`
⇒ l'avocat croit ali innocent.
— `n2 : Négation`, `n2.objet = {karim instance-of AccuséRécidiviste}`
⇒ karim n'est pas récidiviste (fait nié explicitement).

Algorithme d'héritage avec exceptions

Notre moteur `inherits(n, r, t)` effectue un BFS depuis le nœud n : à chaque niveau, si un arc direct $\xrightarrow{r} t$ est trouvé, son signe décide immédiatement (positif ⇒ `INHERITED`, négatif ⇒ `BLOCKED`). Sinon on remonte les arcs `is-a`. C'est le **principe de proximité** : la spécialisation la plus proche prime.

Exploitation

Requête	Résultat	Pourquoi
Accuse presume Innocence ?	**INHERITED**	arc direct sur Accuse
Recidiviste presume Innocence ?	**BLOCKED**	exception ¬presume au niveau Récidiviste
Soolking sous-contrat Label ?	**INHERITED**	hérité via ArtisteSigne
ArtisteIndep sous-contrat Label ?	**BLOCKED**	exception stricte

Implémentation

Package `rcr.semantic`, fichier unique `SemanticNetworkExample.java`. Trois classes internes : `Node` (id, label, type), `Arc` (from, to, relation, style, negated), `SemanticNetwork` (collections de nœuds/arcs + moteur). L'enum `Inheritance.{INHERITED, BLOCKED, NOT_FOUND}` et la méthode `queryRelationWithBlocked()` permettent d'interroger le réseau avec une relation cible et de récupérer simultanément les héritages réussis et les blocages par exception.

9 Tableau comparatif

	Modale	**Défauts**	**Classique**	**Description**	**Rés. Sém.**
Monotone	Oui	Non	Oui	Oui	Non*
Monde ouvert	Non	Non	Non	Oui (OWA)	Non
Exceptions	Non	Oui	Non	Partiel	Oui
Quantificateurs	Non	Oui (FOL)	Oui(FOL)	Oui($\exists, \forall$)	Non
Mondes possibles	Oui	Non	Non	Non	Non
Complexité	PSPACE	Π_2^P	NP (SAT)	ExpTime	P
Outil	Tweety	Tweety	SAT4J	OWL+HermiT	Tweety
Standard	K/T/S4/S5	Reiter 1980	—	OWL 2 (W3C)	—

* avec héritage à exceptions

Table 3 – Comparaison des cinq formalismes

Références

Clé	**Référence**
[Reiter 1980]	R. Reiter, *A Logic for Default Reasoning*, AI, 13(1–2) :81–132
[Baader 2003]	Baader et al., *The Description Logic Handbook*, Cambridge UP
[Shearer 2008]	Shearer et al., *HermiT : A Highly-Efficient OWL Reasoner*, OWLED
[W3C 2009]	W3C OWL Working Group, *OWL 2 Web Ontology Language*, Recommendation
[Horridge 2011]	Horridge & Bechhofer, *The OWL API*, Semantic Web, 2(1) :11–21
[Thimm 2014]	M. Thimm, *Tweety — Java Libraries for KR*, KR, Vienna
[Kripke 1963]	S. A. Kripke, *Semantical Analysis of Modal Logic I*
[Blackburn 2001]	Blackburn et al., *Modal Logic*, Cambridge UP
[Quillian 1968]	M. R. Quillian, *Semantic Memory*, MIT Press
[Le Berre 2010]	Le Berre & Parrain, *The SAT4J Library*, JSAT, 7 :59–64
[Brachman 2004]	Brachman & Levesque, *KR&R*, Morgan Kaufmann

Table 4 – Table des références bibliographiques